

Article

GPT-Based Text-to-SQL for Spatial Databases

Hui Wang ^{1,*}, Li Guo ^{2,†}, Yubin Liang ¹, Le Liu ³ and Jiajin Huang ^{4,5}

¹ Department of Geography, Tianjin Normal University, Tianjin 300387, China; ybliang@tjnu.edu.cn

² Institute of Geospatial Information, Information Engineering University, Zhengzhou 450001, China; gl_750312@163.com

³ Faculty of Architectural Engineering, Tianjin University, Tianjin 300350, China; liule0328@tju.edu.cn

⁴ Faculty of Information Technology, Beijing University of Technology, Beijing 100124, China; jhuang@bjut.edu.cn

⁵ Beijing International Collaboration Base on Brain Informatics and Wisdom Services, Beijing 100124, China

* Correspondence: wanghui@tjnu.edu.cn

† These authors contributed equally to this work.

Abstract

Text-to-SQL for spatial databases enables the translation of natural language questions into corresponding SQL queries, allowing non-experts to easily access spatial data, which has gained increasing attention from researchers. Previous research has primarily focused on rule-based methods. However, these methods have limitations when dealing with complicated or unknown natural language questions. While advanced machine learning models can be trained, they typically require large labeled training datasets, which are severely lacking for spatial databases. Recently, Generative Pre-Trained Transformer (GPT) models have emerged as a promising paradigm for Text-to-SQL tasks in relational databases, driven by carefully designed prompts. In response to the severe lack of datasets for spatial databases, we have created a publicly available dataset that supports both English and Chinese. Furthermore, we propose a GPT-based method to construct prompts for spatial databases, which incorporates geographic and spatial database knowledge into the prompts and requires only a small number of training samples, such as 1, 3, or 5 examples. Extensive experiments demonstrate that incorporating geographic and spatial database knowledge into prompts improves the accuracy of Text-to-SQL tasks for spatial databases. Our proposed method can help non-experts access spatial databases more easily and conveniently.

Keywords: spatial databases; Text-to-SQL; GPT



Academic Editors: Wolfgang Kainz and Eliseo Clementini

Received: 20 May 2025

Revised: 18 July 2025

Accepted: 22 July 2025

Published: 24 July 2025

Citation: Wang, H.; Guo, L.; Liang, Y.; Liu, L.; Huang, J. GPT-Based Text-to-SQL for Spatial Databases. *ISPRS Int. J. Geo-Inf.* **2025**, *14*, 288. <https://doi.org/10.3390/ijgi14080288>

Copyright: © 2025 by the authors. Published by MDPI on behalf of the International Society for Photogrammetry and Remote Sensing. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

According to a study by [1], 80% of big data is spatial data. Spatial databases, which offer highly specialized spatial query and analysis capabilities, have thus become essential tools for managing this data. While experts can interact with these databases using SQL, the language can be challenging for non-experts, limiting their ability to fully leverage this data managed by spatial databases.

Text-to-SQL for spatial databases offers a promising solution by translating the natural language question/text (NLQ/Text) into the corresponding SQL query, as depicted in Figure 1. This enables non-experts to easily access spatial data and has garnered increasing attention from researchers.

Article

基于GPT的空间数据库文本到SQL转换

王辉 ^{1,*†}, 郭力 ^{2†}, 梁宇彬 ¹, 刘乐 ³ 和 黄家晋 ^{4,5}

¹ 天津师范大学地理系, 中国天津 300387; ybliang@tjnu.edu.cn ² 信息工程大学地理空间信息研究所, 中国郑州 450001; gl_750312@163.com ³ 天津大学建筑工程学院, 中国天津 300350; liule0328@tju.edu.cn ⁴ 北京工业大学信息技术学院, 中国北京 100124; jhuang@bjut.edu.cn ⁵ 北京脑信息与智慧服务国际合作基地, 中国北京 100124
* 通信作者: wanghui@tjnu.edu.cn [†] 这些作者对本工作贡献相同。

摘要

面向空间数据库的文本到SQL (Text-to-SQL) 技术能够将自然语言问题转换为相应的SQL查询, 使非专业人士也能轻松访问空间数据, 这一技术已经引起研究人员越来越多的关注。之前的研究主要集中在基于规则的方法。然而, 这些方法在处理复杂或未知的自然语言问题时存在局限性。虽然可以训练先进的机器学习模型, 但它们通常需要大量标注的训练数据, 而空间数据库方面的这类数据严重不足。近年来, 生成式预训练变换器 (GPT) 模型作为关系型数据库Text-to-SQL任务的有希望的范式出现, 其性能依赖于精心设计的提示 (prompts)。针对空间数据库数据集严重缺乏的问题, 我们创建了一个公开可用的数据集, 支持英文和中文。此外, 我们提出了一种基于GPT的方法来为空间数据库构建提示 (prompts), 该方法结合了



学术编辑: 沃尔夫冈·凯恩茨 和 埃利塞奥·克莱门蒂尼

收到: 2025年5月20日
修订: 2025年7月18日
接受: 2025年7月22日
发布: 2025年7月24日

引用: Wang, H.; Guo, L.; Liang, Y.; Liu, L.; Huang, J. 基于 GPT 的空间数据库文本到 SQL 转换。
ISPRS Int. J. Geo-Inf. **2025**, *14*, 288. <https://doi.org/10.3390/ijgi14080288>

版权: © 2025 作者所有。由 MDPI 代表国际摄影测量与遥感学会出版。许可方 MDPI, 瑞士巴塞尔。本文为开放获取文章, 根据知识共享署名 (CC BY) 许可协议 (<https://creativecommons.org/licenses/by/4.0/>) 的条款和条件分发。

关键词: 空间数据库; 文本到SQL; GPT

1. 引言

根据[1]的一项研究, 80%的大数据是空间数据。因此, 空间数据库因提供高度专业化的空间查询和分析能力, 已成为管理这些数据的核心工具。虽然专家可以使用SQL与这些数据库进行交互, 但对非专家来说, 这种语言可能具有挑战性, 从而限制了他们充分利用空间数据库管理的数据的能力。

用于空间数据库的文本到SQL (Text-to-SQL) 通过将自然语言问题/文本 (NLQ/Text) 翻译为对应的SQL查询, 如图1所示, 提供了一种有前景的解决方案。这使非专业人员能够轻松访问空间数据, 并且越来越受到研究人员的关注。

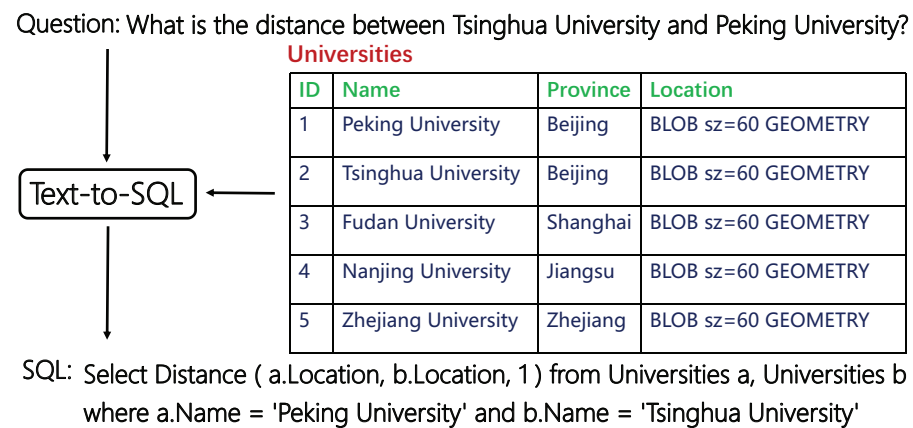


Figure 1. An example of Text-to-SQL for spatial databases.

Previous studies have investigated the problem of translating natural language questions into SQL queries for spatial databases [2–4]. These studies can be summarized as rule-based methods. Although these methods do not require large amounts of training data, they have limitations when dealing with complicated or unknown natural language questions.

These limitations could theoretically be addressed by training a machine learning model. However, the model requires a labeled training dataset that not only covers a diverse range of spatial natural language questions but also includes the corresponding correct SQL queries. Additionally, the dataset must be sufficiently large to ensure that the trained machine learning model achieves satisfactory performance. Unfortunately, due to the complexity of spatial queries, creating such a dataset can be both time-consuming and labor-intensive.

Recently, Generative Pre-Trained Transformer (GPT) models, such as GPT-4 and DeepSeek, have demonstrated impressive performance across a wide range of tasks. These models are designed to take a text-based input, known as a prompt, and generate human-like responses. When using a GPT model to generate an accurate SQL query, one critical factor to consider is the construction of the prompt. The prompt should be based on the natural language question and the corresponding database schema. Studies [5–7] indicate that including examples related to the natural language question in the prompt enables GPT models to generate outputs that better align with user expectations.

In this paper, we propose a GPT-based method to construct prompts for spatial databases, which incorporates geographic and spatial database knowledge into the prompts and requires only a small number of training samples, such as 1, 3, or 5 examples. Additionally, we have created a dataset containing spatial natural language questions paired with their corresponding SQL queries, available in both English and Chinese. This dataset can be used for model training and performance evaluation. Furthermore, our framework requires only a small number of training samples, such as 1, 3, or 5 examples, to effectively guide the GPT model in generating accurate SQL queries.

The remainder of this paper is organized as follows. Section 2 reviews related work on Text-to-SQL tasks for spatial databases. Section 3 presents our proposed method. Section 4 outlines the experimental design used to evaluate our method. Section 5 presents the results of the experiments. Section 6 discusses the limitations of our method. Finally, Section 7 concludes the paper.

2. Related Work

High-quality datasets are essential for studying and evaluating the Text-to-SQL task. Researchers have developed several annotated datasets for Text-to-SQL, including Geo-Query [8], Restaurants [9], Scholar [10], WikiSQL [11], Spider [12], CS Spider [13], DuSQL [14],

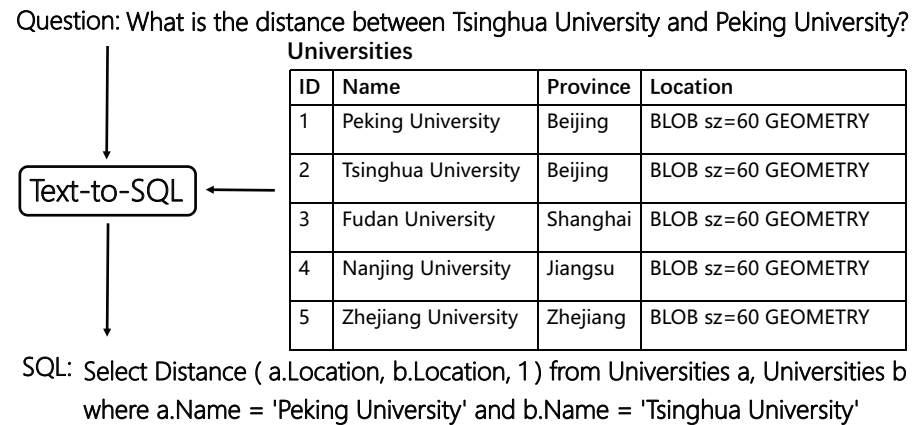


图1. 空间数据库文本到SQL的示例。

以往的研究已经调查了将自然语言问题翻译为空间数据库的 SQL 查询的问题 [2–4]。这些研究可以总结为基于规则的方法。虽然这些方法不需要大量的训练数据，但在处理复杂或未知的自然语言问题时存在局限性。

这些限制理论上可以通过训练一个机器学习模型来解决。然而，该模型需要一个标注的训练数据集，该数据集不仅涵盖各种空间自然语言问题，还包括对应的正确 SQL 查询。此外，数据集必须足够大，以确保训练出的机器学习模型能够达到令人满意的性能。不幸的是，由于空间查询的复杂性，创建这样的数据集既耗时又费力。

最近，生成式预训练变换器（GPT）模型，如 GPT-4 和 DeepSeek，在广泛的任务中表现出了令人印象深刻的性能。这些模型旨在接受一种基于文本的输入，即提示，并生成类似人类的响应。在使用 GPT 模型生成准确的 SQL 查询时，一个关键因素是提示的构建。提示应基于自然语言问题及相应的数据库模式。研究 [5–7] 表明，在提示中包含与自然语言问题相关的示例，可以使 GPT 模型生成的输出更符合用户的期望。

在本文中，我们提出了一种基于GPT的方法来构建空间数据库的提示，该方法将地理和空间数据库知识融入提示中，并且只需少量训练样本，例如1、3或5个示例。此外，我们创建了一个包含空间自然语言问题及其对应SQL查询的数据集，该数据集提供英文和中文版本。该数据集可用于模型训练和性能评估。此外，我们的框架只需少量训练样本，例如1、3或5个示例，就能有效引导GPT模型生成准确的SQL查询。

本文其余部分的组织结构如下。第2节回顾了空间数据库Text-to-SQL任务的相关工作。第3节介绍了我们提出的方法。第4节概述了用于评估我们方法的实验设计。第5节展示了实验结果。第6节讨论了我们方法的局限性。最后，第7节对本文进行总结。

2. 相关工作

高质量的数据集对于研究和评估文本到SQL任务至关重要。研究人员已经开发了几个用于文本到SQL的标注数据集，包括GeoQuery [8]、Restaurants [9]、Scholar [10]、WikiSQL [11]、Spider [12]、CSpider [13]、DuSQL [14]

Spider-DK [15], Spider-Syn [16], and Spider-SSP [17]. These datasets consist of <NLQ/Text, SQL > pairs and are widely utilized in academic research on Text-to-SQL.

To facilitate the movement of Text-to-SQL from academic research to real applications, Alibaba Group has partnered with institutions such as the University of Hong Kong to introduce the BIRD dataset [18]. This dataset focuses on extensive and authentic database content, serving as a large-scale benchmark for databases.

These datasets mentioned above all focus on relational databases. Although the GeoQuery and Restaurants datasets are related to the geographic domain, they still lack the relevant data for spatial queries. Consequently, there is still a significant gap in Text-to-SQL datasets specifically designed for spatial databases.

The advancement of deep learning, along with the availability of large-scale Text-to-SQL datasets, has significantly accelerated research on Text-to-SQL for relational databases. Several methods, including Seq2Seq [19], IRNet [20], RYANSQL [21], and SQLova [22], along with pre-trained models such as TaBERT [23], TaPas [24], and Grappa [25], have been proposed and have demonstrated promising results.

Among these methods, SpatialNLI [26,27] primarily focuses on addressing the semantic ambiguity of spatial entities, such as determining whether ‘Mississippi’ in a natural language question refers to a river or a state. However, the training datasets used by SpatialNLI are still designed for relational databases rather than spatial databases.

Due to the lack of the data of spatial queries in the existing Text-to-SQL datasets, models trained by current Text-to-SQL datasets with deep learning methods often face challenges when applied directly to the Text-to-SQL task for spatial databases. As a result, current research on Text-to-SQL for spatial databases primarily employs rule-based methods [2–4]. This is largely because rule-based methods do not require large-scale training datasets. However, while rule-based methods are suitable for simple user queries, it is challenging, if not impossible, to design SQL templates in advance for various scenarios or domains.

Recently, GPT models have emerged as a new paradigm for the Text-to-SQL task [28–30]. Different from prior studies, the core challenge in GPT-based Text-to-SQL solutions lies in how to guide the model to generate correct SQL queries, specifically how to construct an effective prompt. Many studies have proposed various approaches for prompt construction [4,7,31–33]. However, these approaches are primarily focused on relational databases and do not account for the unique complexities of spatial databases. As a result, directly applying them to the Text-to-SQL task for spatial databases does not yield satisfactory results, as we will demonstrate in our experiments.

The growing demand for spatiotemporal data and modeling in geosciences has greatly advanced geospatial code generation technology. Refs. [34–38] specifically examine the use of large language models for generating geospatial analysis code. For geospatial SQL generation, Ref. [39] briefly reports on the potential of using GPT models for Text-to-SQL tasks targeting spatial databases.

Despite these advancements, methods for constructing prompts to address Text-to-SQL tasks for spatial databases remain limited. In this study, we aim to develop a method of prompt construction for the Text-to-SQL task specifically designed for spatial databases, while simultaneously creating a dedicated dataset for training and evaluating this task.

Spider-DK [15]、Spider-Syn [16] 和 Spider-SSP [17]。这些数据集由 <NLQ/文本、SQL> 对组成，并广泛用于文本到 SQL 的学术研究。

为了促进文本到SQL（Text-to-SQL）技术从学术研究向实际应用的转移，阿里巴巴集团与香港大学等机构合作，推出了BIRD数据集 [18]。该数据集专注于广泛且真实的数据库内容，作为数据库的大规模基准测试数据。

上述提到的这些数据集都专注于关系型数据库。尽管GeoQuery和Restaurants数据集与地理领域相关，但它们仍然缺乏用于空间查询的相关数据。因此，专门为空间数据库设计的文本到SQL数据集仍然存在显著的空白。

深度学习的发展，以及大规模文本到SQL数据集的可用性，显著加快了关系数据库文本到SQL的研究。已经提出了几种方法，包括Seq2Seq [19]、IRNet [20]、RYANSQL [21]和SQLova [22]，以及预训练模型如TaBERT [23]、TaPas [24]和Grappa [25]，并且这些方法已经展示了有希望的结果。

在这些方法中，SpatialNLI [26,27] 主要关注解决空间实体的语义歧义，例如确定自然语言问题中的‘Mississippi’指的是河流还是州。然而，SpatialNLI 使用的训练数据集仍然是为关系数据库而设计的，而不是空间数据库。

由于现有 Text-to-SQL 数据集中缺乏空间查询的数据，使用当前 Text-to-SQL 数据集通过深度学习方法训练的模型在直接应用于空间数据库的 Text-to-SQL 任务时通常会面临挑战。因此，目前针对空间数据库的 Text-to-SQL 研究主要采用基于规则的方法 [2–4]。这在很大程度上是因为基于规则的方法不需要大规模的训练数据集。然而，虽然基于规则的方法适用于简单的用户查询，但对于各种场景或领域，提前设计 SQL 模板是具有挑战性的，甚至是不可能的。

最近，GPT 模型已成为 Text-to-SQL 任务的新范式 [28–30]。与以往的研究不同，基于 GPT 的 Text-to-SQL 解决方案的核心挑战在于如何引导模型生成正确的 SQL 查询，具体来说就是如何构建有效的提示。许多研究已经提出了各种提示构建方法 [4,7,31–33]。然而，这些方法主要集中于关系型数据库，并没有考虑空间数据库的独特复杂性。因此，将它们直接应用于空间数据库的 Text-to-SQL 任务并不能获得令人满意的结果，正如我们的实验将展示的那样。

地球科学中对时空数据和建模的需求日益增长，这极大地推动了地理空间代码生成技术的发展。参考文献 [34–38] 专门研究了使用大型语言模型生成地理空间分析代码的情况。对于地理空间SQL生成，参考文献 [39] 简要报告了使用GPT模型进行面向空间数据库的文本到SQL任务的潜力。

尽管取得了这些进展，用于构建针对空间数据库的 Text-to-SQL 任务提示的方法仍然有限。在本研究中，我们旨在开发一种专门针对空间数据库的 Text-to-SQL 任务的提示构建方法，同时创建一个专门用于训练和评估该任务的数据集。

3. Method

3.1. Overall Framework

Our method consists of two steps: (1) We send the prompt to the GPT model, which generates an SQL query based on the provided prompt. (2) The generated SQL query is then executed in the spatial database management system (SDBMS). If no exceptions occur, the SQL query is considered the final prediction from the GPT model. If an exception is detected, a schema tip (e.g., “the table ‘t’ does not exist in the database and should not be used”) is extracted from the exception feedback and incorporated back into the prompt, after which the process is repeated. The procedure terminates once a predefined repetition threshold is reached.

The prompt in our method consists of six components: (1) SDBMS knowledge, (2) examples of natural language questions similar to the target question along with their corresponding SQL queries, (3) the database schema, (4) a geographic description of each table in the database, (5) schema tips, and (6) the target question.

Figure 2 illustrates the overall framework of our method, while Figure 3 provides a snippet of the prompt. The details of the six components of the prompt are outlined in the following subsections.

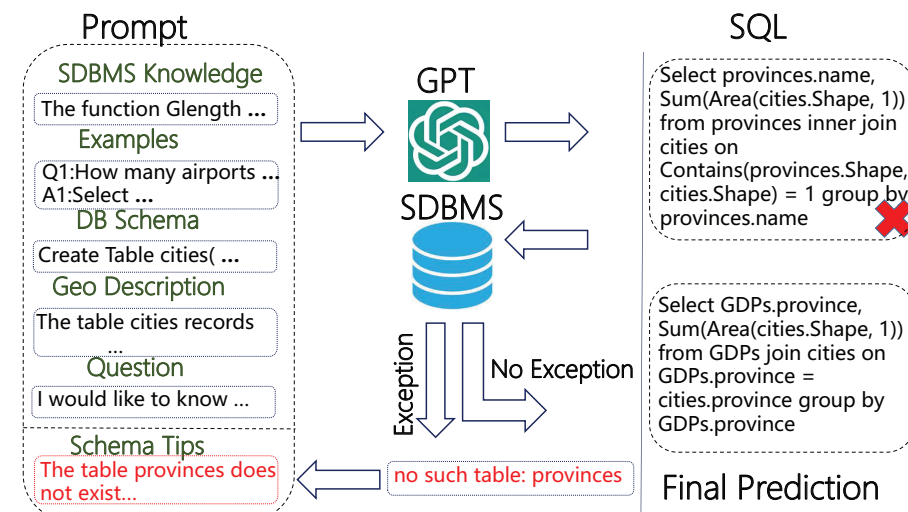


Figure 2. Overall framework of our method.

3.2. SDBMS Knowledge

SDBMSs are typically implemented by means of spatial extensions to relational databases and are offered in several types, such as PostgreSQL with PostGIS, Oracle Spatial, SQL Server Spatial, and SQLite with SpatiaLite. These SDBMSs provide spatial functions to support various spatial queries and analysis capabilities. However, the names of spatial functions corresponding to identical capabilities can differ across various SDBMSs. For instance, when calculating the area of a geometric object, the function is named STArea in SQL Server, whereas in SpatiaLite, the corresponding function is simply designated as Area. Furthermore, within a single SDBMS, the names of these functions may vary across different versions.

These inconsistencies may cause a mismatch between the function names in the SQL query predicted by the GPT model and those provided by the SDBMS used.

Additionally, within the same SDBMS, some spatial functions used for calculations may have different parameterizations, which can result in differing outcomes. For example, in SpatiaLite, the results of Distance (geom1 Geometry, geom2 Geometry, 1) and Distance (geom1 Geometry, geom2 Geometry) differ, with the former providing a more precise result.

3. 方法

3.1. Overall Framework

我们的方法包括两个步骤：(1) 我们将提示发送给GPT模型，该模型根据提供的提示生成SQL查询。(2) 生成的SQL查询随后在空间数据库管理系统（SDBMS）中执行。如果没有发生异常，则该SQL查询被视为GPT模型的最终预测。如果检测到异常，则从异常反馈中提取一个模式提示（例如，“数据库中不存在表‘t’，不应使用该表”），并将其重新整合到提示中，然后重复该过程。一旦达到预定义的重复杂数阈值，程序即终止。

我们方法中的提示包含六个部分：(1) SDBMS 知识，(2) 与目标问题相似的自然语言问题及其对应的 SQL 查询示例，(3) 数据库模式，(4) 数据库中每个表的地理描述，(5) 模式提示，(6) 目标问题。

图2展示了我们方法的整体框架，而图3提供了提示的一个片段。提示的六个组成部分的详细信息将在以下小节中说明。

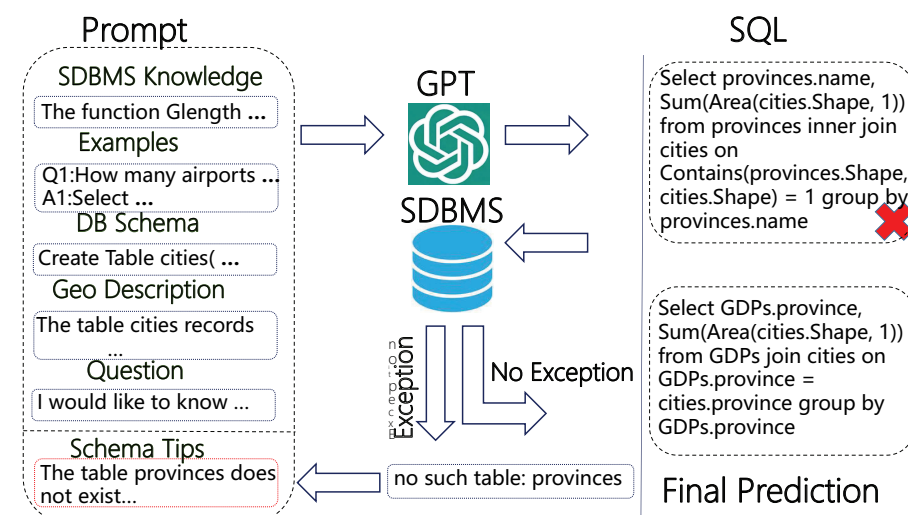


图2. 我们方法的整体框架。

3.2. SDBMS Knowledge

SDBMS 通常通过关系数据库的空间扩展来实现，并且提供多种类型，例如带有 PostGIS 的 PostgreSQL、Oracle Spatial、SQL Server Spatial 以及带有 SpatiaLite 的 SQLite。这些 SDBMS 提供空间函数以支持各种空间查询和分析能力。然而，具有相同功能的空间函数名称在不同的 SDBMS 中可能不同。例如，在计算几何对象的面积时，该函数在 SQL Server 中被命名为 STArea，而在 SpatiaLite 中，对应的函数仅被称为 Area。此外，在单个 SDBMS 中，这些函数的名称在不同版本之间可能会有所不同。

这些不一致可能会导致 GPT 模型预测的 SQL 查询中的函数名称与所使用的 SDBMS 提供的函数名称不匹配。

此外，在同一个 SDBMS 中，用于计算的一些空间函数可能具有不同的参数设置，这可能导致结果不同。例如，在 SpatiaLite 中，Distance (geom1 Geometry, geom2 Geometry, 1) 和 Distance (geom1 Geometry, geom2 Geometry) 的结果不同，前者提供了更精确的结果。

You are an expert in the SpatiLite spatial database and highly skilled in writing SQL queries.

In SpatiLite,

The function Area(s Surface, 1) returns the area of s, measured in meters.

The function GLength(c Curve, 1) returns the length of c, measured in meters.

...

/ Some SQL examples are provided based on similar problems: */*

/ Answer the following: I would like to know the total area of the Beijing-Tianjin-Hebei region. Select Sum(Area(provinces.Shape, 1)) from provinces where provinces.name = '北京市' or provinces.name = '天津市' or provinces.name = '河北省'*

/ Answer the following: Which provinces border Guangdong Province, and what is the area of province? */*

Select b.name , Area(b.Shape, 1) from provinces a inner join provinces b On Touches(a.Shape, b.Shape) 1 where a.name = '广东省'

...

/ Given the following database schema: */*

```
CREATE TABLE 'airports' (
  IATA TEXT PRIMARY KEY,
  runway_number INTEGER,
  airfield_area_class TEXT,
  Name TEXT,
  Location POINT)
```

The 'airports' table records information about various airports in China.

```
CREATE TABLE 'cities' (
  name TEXT PRIMARY KEY,
  administrative_division_code TEXT,
  province TEXT REFERENCES "GDPs" (province),
  Shape MULTIPOLYGON)
```

The 'cities' table records information about various cities in China.

/ Answer the following with no explanation: What is the total area of the three northeastern provinces? The three northeastern provinces refer to Liaoning, Jilin, and Heilongjiang. Liaoning is represented name '辽宁省', Jilin is represented by the name '吉林省', and Heilongjiang is represented by the name '黑龙江省'. The table "provinces" does not exist in the database and should not be used. */*

SELECT

Figure 3. A snippet of the prompt from our method.

To ensure that the GPT model accurately utilizes the functions provided by the SDBMS used and meets the user's specific computational requirements, we first specify the SDBMS used in the prompt. For example, "You are an expert in the SpatiaLite spatial database and highly skilled in writing SQL queries". In our experiments, we use SpatiaLite.

You are an expert in the SpatiaLite spatial database and highly skilled in writing SQL queries.

In SpatiaLite,

The function Area(s Surface, 1) returns the area of s, measured in meters.

The function GLength(c Curve, 1) returns the length of c, measured in meters.

...

/* Some SQL examples are provided based on similar problems: */

/* Answer the following: I would like to know the total area of the Beijing-Tianjin-Hebei region. Select Sum(Area(provinces.Shape, 1)) from provinces where provinces.name = '北京市' or provinces.name = '天津市' or provinces.name = '河北省'

/* Answer the following: Which provinces border Guangdong Province, and what is the area of province? */

Select b.name , Area(b.Shape, 1) from provinces a inner join provinces b On Touches(a.Shape, b.Shape) 1 where a.name = '广东省'

...

/* Given the following database schema: */

```
CREATE TABLE 'airports' (
  IATA TEXT PRIMARY KEY,
  runway_number INTEGER,
  airfield_area_class TEXT,
  Name TEXT,
  Location POINT)
```

The 'airports' table records information about various airports in China.

```
CREATE TABLE 'cities' (
  name TEXT PRIMARY KEY,
  administrative_division_code TEXT,
  province TEXT REFERENCES "GDPS" (province),
  Shape MULTIPOLYGON)
```

The 'cities' table records information about various cities in China.

/* Answer the following with no explanation: What is the total area of the three northeastern provinces? The three northeastern provinces refer to Liaoning, Jilin, and Heilongjiang. Liaoning is represented name '辽宁省', Jilin is represented by the name '吉林省', and Heilongjiang is represented by the name '黑龙江省'. The table "provinces" does not exist in the database and should not be used. */

```
SELECT
```

图 3. 我们方法中提示的一个片段。

为了确保 GPT 模型能够准确使用所使用 SDBMS 提供的功能并满足用户的特定计算需求，我们首先在提示中指定所使用的 SDBMS。例如，“你是 SpatiaLite 空间数据库的专家，并且非常擅长编写 SQL 查询”。在我们的实验中，我们使用 SpatiaLite。

Additionally, we provide clear explanations of some commonly used functions in the prompt, such as GLength, Area, Distance, Centroid, Touches, and Intersects. For example, “The function GLength (c Curve, 1) returns the length of curve c, measured in meters”.

3.3. Examples

We use the Masked Question Similarity Selection method [6,33] to measure the similarity between the target question and questions in the training dataset. Next, we select the K most similar questions, along with their corresponding SQL queries, from the training dataset as examples.

3.4. Database Schema

The database schema defines the structure of the database, including tables, columns, relationships between tables, data types, and additional elements. In this section, we use a Code Representation [40,41] to describe the corresponding database schema. Specifically, as shown in Figure 3, it directly presents the “CREATE TABLE” SQL statements.

3.5. Geographic Descriptions

In Text-to-SQL tasks, adding description for each table in the prompt may help clarify table semantics and reduce ambiguity. In this section, we add a geographic region description for each table in the prompt. This is based on our observation that, in spatial databases, the data in each table typically corresponds to a specific geographic region. For example, in the Traffic database we use, the ‘roads’ table contains information about roads in Nanjing. There is a category of questions, such as “What is the total length of each road in Nanjing?” and “What is the total length of each road?”, which we refer to as questions related to regions. These two questions are essentially equivalent in meaning. However, when faced with the question “What is the total length of each road in Nanjing?”, the GPT model often misinterprets ‘Nanjing’ as a column value in the database, leading to the generation of an incorrect SQL query.

To address this issue, we explicitly describe the geographic region associated with each table in the prompt. For instance, for the ‘roads’ table, we include the description “The ‘roads’ table records information about roads in Nanjing.”.

3.6. Question

In this section, we adopt a simple question representation in DAIL [33], where the natural language question is prefixed with “Answer the following with no explanation:” and is followed by “SELECT ”. When generating SQL queries, the GPT model sometimes provides explanations for certain terms. The phrase “with no explanation” helps ensure that GPT only returns the answer.

3.7. Schema Tips

In Text-to-SQL tasks, self-correction with execution feedback may help improve prediction accuracy. In this section, we focus on handling three types of feedback: nonexistent table names, nonexistent column names, and ambiguous column names. This is because foundational data (e.g., administrative divisions or roads) often appears across multiple spatial databases. Consequently, table or column names in the examples may share identical or similar names or meanings with those in the target question. As shown in Figure 3, the term ‘provinces’ appears in both the examples and the target question. Such overlap can lead the GPT model to generate SQL queries referencing tables or columns from the examples that do not exist in the given database schema.

此外，我们提供了对提示中一些常用函数的清晰解释，例如 GLength、Area、Distance、Centroid、Touches 和 Intersects。例如，“函数 GLength(c Curve, 1) 返回曲线 c 的长度，单位为米。”

3.3. Examples

我们使用掩码问题相似性选择方法 [6,33] 来衡量目标问题与训练数据集中问题之间的相似性。接下来，我们从训练数据集中选择 K 个最相似的问题及其对应的 SQL 查询作为示例。

3.4. Database Schema

数据库模式定义了数据库的结构，包括表、列、表之间的关系、数据类型和其他元素。在本节中，我们使用代码表示法 [40,41] 来描述相应的数据库模式。具体来说，如图3所示，它直接呈现了“CREATE TABLE” SQL 语句。

3.5. Geographic Descriptions

在 Text-to-SQL 任务中，在提示中为每个表添加描述可能有助于澄清表的语义并减少歧义。在本节中，我们在提示中为每个表添加地理区域描述。这是基于我们的观察：在空间数据库中，每个表中的数据通常对应于特定的地理区域。例如，在我们使用的交通数据库中，‘roads’ 表包含南京的道路信息。有一类问题，例如“南京每条道路的总长度是多少？”和“每条道路的总长度是多少？”，我们称之为与区域相关的问题。这两个问题在本质上是等价的。然而，当面临“南京每条道路的总长度是多少？”这个问题时，GPT 模型经常会将‘南京’误解为数据库中的一个列值，从而生成错误的 SQL 查询。

为了解决这个问题，我们在提示中明确描述与每个表相关的地理区域。例如，对于“roads”表，我们包括描述“‘roads’表记录南京的道路信息。”

3.6. Question

在本节中，我们采用 DAIL [33] 中的简单问题表示方法，其中自然语言问题前缀为“Answer the following with no explanation:”，后跟“SELECT”。在生成 SQL 查询时，GPT 模型有时会对某些术语提供解释。短语“with no explanation”有助于确保 GPT 仅返回答案。

3.7. Schema Tips

在文本到SQL的任务中，利用执行反馈进行自我纠正可能有助于提高预测准确性。在本节中，我们重点处理三种类型的反馈：不存在的表名、不存在的列名以及模糊的列名。这是因为基础数据（例如行政区划或道路）经常出现在多个空间数据库中。因此，示例中的表或列名称可能与目标问题中的名称或含义相同或相似。如图3所示，术语“provinces”同时出现在示例和目标问题中。这种重叠可能导致GPT模型生成引用示例中表或列的SQL查询，而这些表或列在给定的数据库模式中并不存在。

To enable the GPT model to correctly reference tables or columns based on the given database schema, we extract three types of schema tips from the exception feedback in Step (2) and incorporate them into the prompt.

- When the exception feedback is “no such table: t”, we extract the table name ‘t’ and append the following sentence, “The table ‘t’ does not exist in the database and should not be used.”, to the prompt.
- When the exception feedback is “no such column: c”, we extract the column name ‘c’ and its corresponding table name ‘t’, and append the following sentence, “The column ‘c’ does not exist in the table ‘t’ and should not be used.”, to the prompt.
- When the exception feedback is “ambiguous column name: c”, we append the following sentence, “Columns in multiple tables may have the same name, so ambiguity should be avoided.”, to the prompt.

4. Experiment

4.1. Experiment Dataset

Due to the limited availability of datasets for spatial databases in Text-to-SQL tasks, we have developed a dedicated dataset to evaluate our proposed method. Datasets for relational databases, such as BIRD [18] and Spider [12], which are among the most widely used, are typically built using SQLite, the world’s most popular database engine. To maintain consistency, our dataset is built on SpatiaLite, which extends SQLite with comprehensive Spatial SQL capabilities.

The developed dataset comprises four types of spatial databases: Ada (Administrative Divisions), Edu (Education), Tourism, and Traffic. All data is sourced from real-world information in China.

In some datasets for relational databases, such as BIRD [18], a Text-to-SQL pair consists of both a text and an SQL query. The text not only includes the question but also contains evidence, which refers to supplementary knowledge typically provided by experts. To ensure that our dataset supports both Chinese and English questions, we have added a ‘name’ field to each Text-to-SQL pair, which specifies the actual values of certain phrases in the database.

For example, in the question “What is the total area of the three northeastern provinces? The three northeastern provinces refer to Liaoning, Jilin, and Heilongjiang.”, the phrase ‘Liaoning’ corresponds to ‘辽宁省’ (in Chinese) in the database, ‘Jilin’ corresponds to ‘吉林省’ (in Chinese), and ‘Heilongjiang’ corresponds to ‘黑龙江省’ (in Chinese). Hence, in the ‘name’ field, we include the description “Liaoning is represented by the name ‘辽宁省’, Jilin is represented by the name ‘吉林省’, and Heilongjiang is represented by the name ‘黑龙江省’”.

Additionally, we refer to the procedure of CSpider [13], a Chinese dataset translated from the English dataset Spider [12], to provide Chinese equivalents for the fields of question, evidence, and name. The details are outlined in Table 1.

In spatial databases, certain natural language questions can be performed using SQL with either non-spatial or spatial columns. For instance, the ‘rivers’ table in the Ada database contains nine columns, including a non-spatial column ‘length’ and a spatial column ‘shape’, the latter being of ‘geometry’ type. To query the length of a river, you can directly access the ‘length’ column or calculate it using the spatial function ‘GLength (shape, 1)’. Similarly, the ‘cities’ table in the Edu database contains 10 columns, including a non-spatial column ‘province’ and a spatial column ‘shape’, while the ‘provinces’ table in the Edu database contains four columns, including a spatial column ‘shape’. To determine the province of a city, you can query the ‘province’ column in the ‘cities’ table, or use the

为了使 GPT 模型能够根据给定的数据库 schema 正确引用表或列，我们从步骤 (2) 的异常反馈中提取三种类型的 schema 提示，并将它们整合到提示中。

- 当异常反馈为“no such table: t”时，我们提取表名‘t’，并在提示中附加以下句子：“数据库中不存在表‘t’，不应使用该表。”
- 当异常反馈为“no such column: c”时，我们提取列名‘c’及其对应的表名‘t’，并将以下句子附加到提示中：“列‘c’在表‘t’中不存在，不应使用。”
- 当异常反馈为“模糊的列名: c”时，我们在提示中追加以下句子：“多个表中的列可能有相同的名称，因此应避免歧义。”

4. 实验

4.1. Experiment Dataset

由于用于文本到 SQL 任务的空间数据库数据集有限，我们开发了一个专门的数据集来评估我们提出的方法。用于关系数据库的数据集，如 BIRD [18] 和 Spider [12]，是最常用的数据集之一，通常使用世界上最流行的数据库引擎 SQLite 构建。为了保持一致性，我们的数据集建立在 SpatiaLite 之上，SpatiaLite 扩展了 SQLite 的功能，提供全面的空间 SQL 能力。

该开发的数据集包含四种类型的空间数据库：Ada（行政区划）、Edu（教育）、旅游和交通。所有数据均来源于中国的真实信息。

在一些用于关系数据库的数据集中，例如 BIRD [18]，一个 Text-to-SQL 对包括文本和 SQL 查询。文本不仅包含问题，还包含证据，证据指的是通常由专家提供的补充知识。为了确保我们的数据集支持中文和英文问题，我们在每个 Text-to-SQL 对中添加了一个“name”字段，该字段指定数据库中某些短语的实际值。

例如，在问题“东北三省的总面积是多少？东北三省指辽宁省、吉林省和黑龙江省。”中，短语‘Liaoning’在数据库中对对应‘辽宁省’，‘Jilin’对应‘吉林省’，‘Heilongjiang’对应‘黑龙江省’。因此，在‘name’字段中，我们包括描述“辽宁由名称‘辽宁省’表示，吉林由名称‘吉林省’表示，黑龙江由名称‘黑龙江省’表示”。

此外，我们参考了 CSpider [13] 的方法，这是从英文数据集 Spider [12] 翻译而来的中文数据集，用于为问题、证据和名称的字段提供中文对应。具体细节列在表 1 中。

在空间数据库中，某些自然语言问题可以使用包含非空间列或空间列的 SQL 来执行。例如，Ada 数据库中的“rivers”表包含九列，包括一个非空间列“length”和一个空间列“shape”，后者是“geometry”类型。要查询一条河流的长度，可以直接访问“length”列或使用空间函数“GLength(shape, 1)”来计算。类似地，Edu 数据库中的“cities”表包含十列，包括一个非空间列“province”和一个空间列“shape”，而 Edu 数据库中的“provinces”表包含四列，包括一个空间列“shape”。要确定一个城市所在的省份，可以查询“cities”表中的“province”列，或者使用

spatial function ‘Within (cities.shape, provinces.shape)’ to determine if the city lies within a specific province.

Table 1. A Text-to-SQL pair from our developed dataset, where the fields ‘questionCHI’, ‘evidenceCHI’, and ‘nameCHI’ represent the Chinese equivalents of the fields ‘question’, ‘evidence’, and ‘name’, respectively.

question: What is the total area of the three northeastern provinces?
 questionCHI: 东三省的总面积是多少?
 evidence: The three northeastern provinces refer to Liaoning, Jilin, and Heilongjiang.
 evidenceCHI: 东三省指辽宁省, 吉林省, 黑龙江省.
 name: Liaoning is represented by the name ‘辽宁省’, Jilin is represented by the name ‘吉林省’, and Heilongjiang is represented by the name ‘黑龙江省’.
 nameCHI: 辽宁省以‘辽宁省’为名称表示, 吉林省以‘吉林省’为名称表示, 黑龙江省以‘黑龙江省’为名称表示.
 SQL: Select Sum (Area (Shape, 1)) from cities where province in (‘辽宁省’, ‘吉林省’, ‘黑龙江省’)

In practical applications, we can calculate the similarity between phrases in the ‘SQL’ field and the corresponding column values in the database, selecting the value with the highest similarity as the actual value in the ‘SQL’ field.

We call these non-spatial columns, such as ‘length’ and ‘province’, whose values can be inferred or calculated from spatial columns or functions, as derived columns. To enhance our dataset and fully evaluate the performance of our method, we transform the Ada, Edu, and Traffic databases by retaining the derived columns in one version and removing them in another. This process results in two versions of each database: Ada¹, Edu¹, Traffic¹ (with derived columns) and Ada², Edu², Traffic² (without derived columns).

We refer to the spatial questions in [2,3] (each containing 100 questions, with the spatial databases and corresponding SQL queries not publicly available), as well as the general questions in BIRD and Spider, to design a set of 200 questions. Within each database type (Ada¹ and Ada², Edu¹ and Edu², Traffic¹ and Traffic²), the questions are identical. In the Ada¹, Edu¹, and Traffic¹ databases, due to the presence of derived columns, a spatial question can be addressed by multiple SQL queries, using either non-spatial columns (general SQL queries) or spatial columns (spatial SQL queries). In contrast, in the Ada², Edu², and Traffic² databases, where derived columns have been removed, spatial questions require spatial columns (i.e., spatial SQL queries). To enhance our dataset, we provide multiple SQL queries for each of the 200 questions whenever possible, resulting in a total of 296 unique SQL queries. The details of the developed dataset are described in Table 2.

Table 2. Detailed statistics for our developed dataset.

Name	Tables	Columns	PKs	FKs	Qs	Rs	SQLs
Ada ¹	6	53	6	1	56	7	90
Ada ²	6	49	6	0	56	7	56
Edu ¹	3	26	3	3	44	0	93
Edu ²	3	22	3	0	44	0	44
Tourism	6	57	6	2	33	1	33
Traffic ¹	11	55	11	4	67	26	81
Traffic ²	11	51	11	3	67	26	67

In the table, the horizontal axis indicates the number of tables, columns, primary keys (PKs), foreign keys (FKs), questions (Qs), questions related to geographic regions (Rs, such as the one “What is the total length of each road in Nanjing?” in Section 3.5), and SQL queries in each database.

空间函数 ‘Within (cities.shape, provinces.shape)’ 用于确定城市是否位于特定省份内。

表 1. 我们开发的数据集中一个文本到 SQL 的示例，其中字段 ‘questionCHI’、‘evidenceCHI’ 和 ‘nameCHI’ 分别表示字段 ‘question’、‘evidence’ 和 ‘name’ 的中文对应项。

question:	What is the total area of the three northeastern provinces?
questionCHI:	东三省的总面积是多少?
evidence:	The three northeastern provinces refer to Liaoning, Jilin, and Heilongjiang.
evidenceCHI:	东三省指辽宁省, 吉林省, 黑龙江省.
name:	Liaoning is represented by the name ‘辽宁省’, Jilin is represented by the name ‘吉林省’, and Heilongjiang is represented by the name ‘黑龙江省’.
nameCHI:	辽宁省以‘辽宁省’为名称表示, 吉林省以‘吉林省’为名称表示, 黑龙江省以‘黑龙江省’为名称表示.
SQL:	Select Sum (Area (Shape, 1)) from cities where province in (‘辽宁省’, ‘吉林省’, ‘黑龙江省’)

在实际应用中，我们可以计算 ‘SQL’ 字段中的短语与数据库中相应列值之间的相似度，选择相似度最高的值作为 ‘SQL’ 字段中的实际值。

我们将这些非空间列称为派生列，例如“长度”和“省份”，其值可以从空间列或函数中推导或计算得出。为了增强我们的数据集并全面评估我们方法的性能，我们对Ada、Edu和Traffic数据库进行转换，在一个版本中保留派生列，在另一个版本中删除它们。这个过程产生了每个数据库的两个版本：含派生列的Ada¹、Edu¹、Traffic¹ ()，以及不含派生列的Ada²、Edu²、Traffic² ()。

我们参考了文献[2,3]中的空间问题（每篇包含100个问题，且空间数据库及相应的SQL查询未公开），以及BIRD和Spider中的一般问题，设计了一套200个问题。在每种数据库类型中（Ada¹和Ada²，Edu¹和Edu²，Traffic¹和Traffic²），问题是相同的。在Ada¹、Edu¹和Traffic¹数据库中，由于存在派生列，空间问题可以通过多条SQL查询解决，既可以使用非空间列（一般SQL查询），也可以使用空间列（空间SQL查询）。相比之下，在Ada²、Edu²和Traffic²数据库中，由于派生列已被移除，空间问题需要使用空间列（即空间SQL查询）。为了增强我们的数据集，我们尽可能为每个200个问题提供多条SQL查询，总共产生了296条独特的SQL查询。开发的数据集的详细信息

表 2. 我们开发的数据集的详细统计信息。

Name	Tables	Columns	PKs	FKs	Qs	Rs	SQLs
Ada ¹	6	53	6	1	56	7	90
Ada ²	6	49	6	0	56	7	56
Edu ¹	3	26	3	3	44	0	93
Edu ²	3	22	3	0	44	0	44
Tourism	6	57	6	2	33	1	33
Traffic ¹	11	55	11	4	67	26	81
Traffic ²	11	51	11	3	67	26	67

在表格中，横轴表示每个数据库中的表格数量、列数、主键 (PK)、外键 (FK)、问题 (Qs)、与地理区域相关的问题 (Rs，例如第3.5节中的“南京每条道路的总长度是多少？”)，以及SQL查询数量。

4.2. Experiment Methods

We conduct an experimental evaluation using the following methods based on the GPT model:

- **SSpa:** We have named our method SSpa, which consists of six components. Among them, the examples, database schema, and question are common components of GPT-based methods for relational databases in Text-to-SQL tasks, while the SDBMS knowledge, geographic descriptions, and schema tips represent the innovative aspects of our method.
- **DAIL-SQL:** DAIL-SQL, developed by Alibaba Group, is a highly effective and efficient method designed to optimize the application of GPT models in Text-to-SQL tasks. Notably, equipped with GPT-4, DAIL-SQL achieved first place on the Spider leaderboard with an execution accuracy of 86.6%, setting a new benchmark. DAIL-SQL is an excellent representative of GPT-based methods for relational databases in Text-to-SQL tasks, consisting mainly of examples, database schema, and question. Hence, we use DAIL-SQL as our baseline method.
- **SSpa-SDBMS:** Compared to the SSpa method, SSpa-SDBMS removes the SDBMS knowledge component while retaining the geographic descriptions and schema tips.
- **SSpa-Geo:** Compared to the SSpa method, SSpa-Geo removes the geographic descriptions component while retaining the SDBMS knowledge and schema tips.
- **SSpa-Tips:** Compared to the SSpa method, SSpa-Tips removes the schema tips component while retaining the SDBMS knowledge and geographic descriptions.

To ensure a fair comparison, we employ GPT-4 for all methods evaluated in this study, given that DAIL-SQL achieves the best performance with GPT-4. To enhance the robustness and reliability of our method, we conduct supplementary comparative experiments using DeepSeek. All methods are accessed through the OpenAI API.

To eliminate any influence from randomness, we set the temperature parameter to 0 by default for both GPT-4 and DeepSeek. Furthermore, the maximum number of repetitions in Step (2) is limited to five. Regardless of whether the last predicted SQL query encounters an execution exception, it is considered the final prediction.

4.3. Evaluation Metrics

The most commonly used evaluation metrics in Text-to-SQL tasks are exact match (EM) and execution accuracy (EX). For EM, the evaluation is based on whether every part of the predicted SQL query matches exactly with the ground truth SQL query. The prediction is considered correct only if all parts of the query are an exact match. For EX, it evaluates the prediction by comparing the execution results of the predicted SQL query and the ground truth SQL query on the same database. A prediction is considered correct only if the execution results are identical.

In spatial databases, the presence of derived columns introduces a challenge where a given question might have multiple valid SQL queries. Additionally, GPT models often exhibit their own distinct style in writing SQL queries, which can result in different SQL queries for the same question. Therefore, when evaluating the methods for spatial databases in Text-to-SQL tasks, the use of the EM metric might be biased, as it requires an exact match of SQL syntax. In contrast, using EX, which directly compares the execution results, is more meaningful and practical. As a result, in our experiments, we exclusively use EX as the evaluation metric.

EX is defined as a percentage, which can be represented as

$$EX = \frac{|\text{Correctly predicted}|}{|\text{All predicted}|} \times 100\%$$

4.2. Experiment Methods

我们使用基于 GPT 模型的以下方法进行实验评估：

- **SSpa**：我们将我们的方法命名为SSpa，它由六个组成部分构成。其中，示例、数据库模式和问题是基于GPT的关系数据库Text-to-SQL任务方法的常见组成部分，而SDBMS知识、地理描述和模式提示则代表了我们的创新方面。
- **DAIL-SQL**：DAIL-SQL 由阿里巴巴集团开发，是一种高效且有效的方法，旨在优化 GPT 模型在 Text-to-SQL 任务中的应用。值得注意的是，配备 GPT-4 的 DAIL-SQL 在 Spider 排行榜上以 86.6% 的执行准确率获得第一名，创下了新的基准。DAIL-SQL 是基于 GPT 的关系数据库 Text-to-SQL 方法的优秀代表，主要由示例、数据库模式和问题组成。因此，我们将 DAIL-SQL 作为我们的基线方法。
- **SSpa-SDBMS**：与SSpa方法相比，SSpa-SDBMS去除了SDBMS知识组件，同时保留地理描述和模式提示。
- **SSpa-Geo**：与SSpa方法相比，SSpa-Geo移除了地理描述组件，同时保留了SDBMS知识和模式提示。
- **SSpa-Tips**：与SSpa方法相比，SSpa-Tips移除了模式提示组件，同时保留了SDBMS知识和地理描述。

为了确保公平的比较，鉴于 DAIL-SQL 在 GPT-4 上取得最佳性能，我们在本研究中评估的所有方法都使用 GPT-4。为了增强我们方法的稳健性和可靠性，我们使用 DeepSeek 进行了补充对比实验。所有方法均通过 OpenAI API 访问。

为了消除任何来自随机性的影响，我们默认将温度参数设置为0，适用于GPT-4和DeepSeek。此外，第（2）步中重复的最大次数限制为五次。无论最后预测的SQL查询是否遇到执行异常，它都被视为最终预测。

4.3. Evaluation Metrics

在 Text-to-SQL 任务中最常用的评估指标是精确匹配（EM）和执行准确性（EX）。对于 EM，评估是基于预测的 SQL 查询的每个部分是否与真实的 SQL 查询完全匹配。只有当查询的所有部分完全匹配时，预测才被认为是正确的。对于 EX，它通过比较预测 SQL 查询和真实 SQL 查询在同一数据库上的执行结果来评估预测。只有当执行结果完全相同时，预测才被认为是正确的。

在空间数据库中，派生列的存在带来了一个挑战，即对于一个给定的问题可能存在多个有效的 SQL 查询。此外，GPT 模型在编写 SQL 查询时通常会表现出自己独特的风格，这可能导致对于相同的问题生成不同的 SQL 查询。因此，在评估空间数据库在 Text-to-SQL 任务中的方法时，使用 EM 指标可能存在偏差，因为它要求 SQL 语法完全匹配。相比之下，使用 EX 指标直接比较执行结果，更有意义且更实用。因此，在我们的实验中，我们仅使用 EX 作为评估指标。

EX 被定义为一个百分比，可以表示为

$$EX = \frac{|\text{Correctly predicted}|}{|\text{All predicted}|} \times 100\%$$

5. Results

We conduct two sets of experiments based on our developed spatial dataset. In the first set of experiments, we create Dataset 1 by combining the Ada¹, Edu¹, Tourism, and Traffic¹ databases along with their respective samples. Initially, we train on Ada¹ and Edu¹ (100 questions and their corresponding SQL queries) and test on Tourism and Traffic¹ (100 questions and their corresponding ground truth SQL queries), applying all prediction methods. We then reverse the roles, training on Tourism and Traffic¹ (100 questions and their corresponding SQL queries) and testing on Ada¹ and Edu¹ (100 questions and their corresponding ground truth SQL queries). Finally, we evaluate all 200 predictions in Dataset 1 using EX. In this context, training means selecting the top-K questions and their corresponding SQL queries from the training dataset that are most similar to the target question to serve as examples in the prompt.

The second set of experiments follows the same procedure but uses Ada², Edu², Tourism, and Traffic² to form Dataset 2. Specifically, we alternate between training on Ada² and Edu² (testing on Tourism and Traffic²) and training on Tourism and Traffic² (testing on Ada² and Edu²), again assessing all 200 predictions in Dataset 2.

Although the questions in Dataset 1 and Dataset 2 are identical, the removal of derived columns and related foreign keys in Dataset 2 leads to significant changes in the schemas of the Ada, Edu, and Traffic databases.

5.1. Performance Evaluation of Each Method

For all methods, we conduct experiments with GPT-4 and DeepSeek on both Dataset 1 and Dataset 2, testing scenarios that include no examples in the prompt, as well as scenarios with 1, 3, or 5 examples included. We refer to the scenario without examples in the prompt as 0-shot, while the scenarios with 1, 3, and 5 examples are labeled as 1-shot, 3-shot, and 5-shot, respectively. The results of these experiments are summarized in Tables 3 and 4.

Table 3. Performance (%) of each method with GPT-4.

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	44.5	52.5	59.5	65.5	34.0	47.0	59.0	57.5
SSpa	85.0	83.5	85.0	87.5	78.0	80.0	81.0	79.0
SSpa-SDBMS	50.0	56.0	61.5	70.5	38.0	48.0	65.5	73.0
SSpa-Geo	80.5	77.5	78.0	77.5	75.0	71.0	71.5	68.5
SSpa-Tips	79.0	80.0	83.5	82.5	74.5	77.5	78.0	75.5

Table 4. Performance (%) of each method with DeepSeek.

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	50.5	63	65.5	70.5	38.0	57.5	62.5	65.5
SSpa	82.5	83	82.5	86	75.5	81	81	81
SSpa-SDBMS	51.5	65	72	78.5	41	65.5	76.5	77.5
SSpa-Geo	79.5	79	79	79	72	76	74.5	73.5
SSpa-Tips	80.5	81	80.5	83.5	68.5	77	78.5	76.5

First, we compare the performance differences between the methods SSpa, SSpa-SDBMS, SSpa-Geo, SSpa-Tips, and DAIL-SQL. As illustrated in Tables 3 and 4, the methods SSpa, SSpa-SDBMS, SSpa-Geo, and SSpa-Tips consistently outperform the baseline method DAIL-SQL across both Dataset 1 and Dataset 2, whether using GPT-4 or DeepSeek. Specifically, the SSpa method with GPT-4 achieves the highest performance of 87.5% on Dataset 1

5. 结果

我们基于开发的空数据数据集进行了两组实验。在第一组实验中，我们通过结合Ada¹、Edu¹、Tourism和Traffic¹数据库及其各自样本创建了数据集1。最初，我们在Ada¹和Edu¹ (的100个问题及其对应的SQL查询)上进行训练，并在Tourism和Traffic¹的100个问题及其对应的真实SQL查询)上进行测试，应用所有预测方法。然后我们交换角色，在Tourism和Traffic¹ (的100个问题及其对应的SQL查询)上进行训练，并在Ada¹和Edu¹ (的100个问题及其对应的真实SQL查询)上进行测试。最后，我们使用EX评估数据集1中所有200个预测。在此上下文中，训练指的是从训练数据集中选择与目标问题最相似的前K个问题及其对应的SQL查询作为提示中的示例。

第二组实验遵循相同的程序，但使用Ada²、Edu²、Tourism和Traffic²来形成数据集2。具体来说，我们交替进行：在Ada²和Edu² (上训练，在Tourism和Traffic²)上测试；以及在Tourism和Traffic² (上训练，在Ada²和Edu²)上测试，再次评估数据集2中的所有200个预测。

虽然数据集1和数据集2中的问题是相同的，但在数据集2中删除派生列和相关外键导致Ada、Edu和Traffic数据库的架构发生了显著变化。

5.1. Performance Evaluation of Each Method

对于所有方法，我们在数据集1和数据集2上对GPT-4和DeepSeek进行了实验，测试的场景包括提示中没有示例的情况，以及包含1个、3个或5个示例的情况。我们将提示中没有示例的场景称为零样本 (0-shot)，而包含1个、3个和5个示例的场景分别标记为一样本 (1-shot)、三样本 (3-shot) 和五样本 (5-shot)。这些实验的结果总结在表3和表4中。

表3. 各方法在GPT-4上的性能 (%)。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	44.5	52.5	59.5	65.5	34.0	47.0	59.0	57.5
SSpa	85.0	83.5	85.0	87.5	78.0	80.0	81.0	79.0
SSpa-SDBMS	50.0	56.0	61.5	70.5	38.0	48.0	65.5	73.0
SSpa-Geo	80.5	77.5	78.0	77.5	75.0	71.0	71.5	68.5
SSpa-Tips	79.0	80.0	83.5	82.5	74.5	77.5	78.0	75.5

表4. 各方法在DeepSeek下的性能 (%)

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	50.5	63	65.5	70.5	38.0	57.5	62.5	65.5
SSpa	82.5	83	82.5	86	75.5	81	81	81
SSpa-SDBMS	51.5	65	72	78.5	41	65.5	76.5	77.5
SSpa-Geo	79.5	79	79	79	72	76	74.5	73.5
SSpa-Tips	80.5	81	80.5	83.5	68.5	77	78.5	76.5

首先，我们比较方法SSpa、SSpa-SDBMS、SSpa-Geo、SSpa-Tips和DAIL-SQL之间的性能差异。如表3和表4所示，无论使用GPT-4还是DeepSeek，方法SSpa、SSpa-SDBMS、SSpa-Geo和SSpa-Tips在数据集1和数据集2上始终优于基线方法DAIL-SQL。具体来说，使用GPT-4的SSpa方法在数据集1上取得了最高的87.5%性能。

in the 5-shot scenario and 81% on Dataset 2 in the 3-shot scenario, surpassing DAIL-SQL by 22% in both scenarios. Similarly, the SSpa method with DeepSeek attains a peak performance of 86% on Dataset 1 in the 5-shot scenario and 81% on Dataset 2, outperforming DAIL-SQL by at least 15.5% in both scenarios. These performance differences align with our expectations, as DAIL-SQL is specifically designed for relational databases, while spatial databases have their own unique characteristics. The SSpa method and its variants (SSpa-SDBMS, SSpa-Geo, and SSpa-Tips) enhance performance by integrating specific knowledge about the used spatial database management, geographic descriptions, or schema tips into the model's prompt, thereby improving their effectiveness in Text-to-SQL tasks for spatial database.

Next, we examine the performance differences of the SSpa and DAIL-SQL methods across Dataset 1 and Dataset 2. As shown in Tables 3 and 4, both methods (SSpa and DAIL-SQL) consistently perform better on Dataset 1 than on Dataset 2 across all scenarios (0-shot, 1-shot, 3-shot, and 5-shot), whether using GPT-4 or DeepSeek. This result can be attributed to the differences in SQL queries between Dataset 1 and Dataset 2. In Dataset 1, due to the presence of derived columns, the questions can be addressed using both general SQL queries and spatial SQL queries. However, in Dataset 2, where derived columns are removed, the questions can only be answered through spatial SQL queries, which are generally more challenging than general SQL queries. Additionally, compared to relational databases, spatial databases provide less available knowledge, causing GPT models to leverage less spatial database knowledge during training, potentially leading to poorer performance in generating spatial SQL queries than general SQL queries. This highlights that, despite the availability of many GPT-based Text-to-SQL methods for relational databases, significant challenges remain when applying them to Text-to-SQL tasks for spatial databases.

Finally, we evaluate the performance of the SSpa method across various scenarios (0-shot, 1-shot, 3-shot, and 5-shot). As shown in Figure 4, for both GPT-4 and DeepSeek, SSpa achieves optimal performance in non-0-shot scenarios, suggesting that incorporating relevant examples in the prompt can enhance SQL query prediction accuracy. However, increasing the number of examples does not consistently improve performance. For instance, with GPT-4, SSpa outperforms 1-shot with 0-shot on Dataset 1 and 5-shot with 3-shot on Dataset 2. The former may result from a poorly chosen or unrepresentative single example, while the latter could stem from interference among examples in the 5-shot scenario. In contrast, DeepSeek does not exhibit these anomalies, likely due to differences in model architecture and training. Regardless, both GPT-4 and DeepSeek achieve their best results in non-0-shot scenarios.

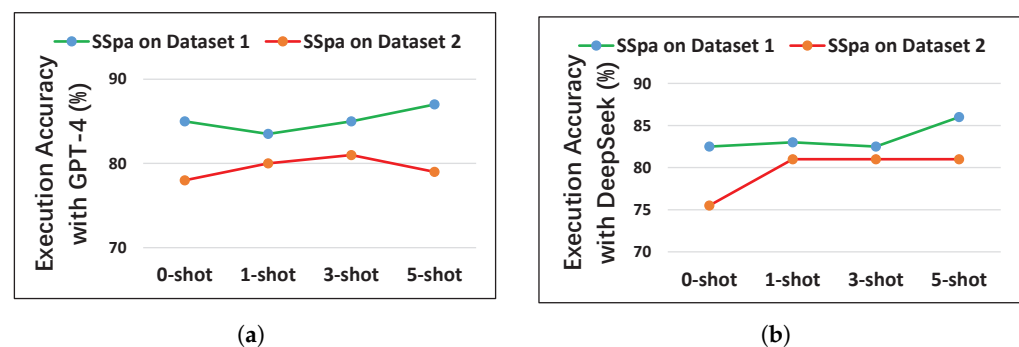


Figure 4. A line chart providing a clear visualization of the performance of SSpa with 0-shot, 1-shot, 3-shot, and 5-shot on both Dataset 1 and Dataset 2. (a) Performance of SSpa with GPT-4. (b) Performance of SSpa with DeepSeek.

在5-shot场景中以及在3-shot场景中，Dataset 2上达到81%，在两个场景中都比DAIL-SQL高出22%。类似地，使用DeepSeek的SSpa方法在5-shot场景中Dataset 1上达到最高性能86%，在Dataset 2上达到81%，在两个场景中至少比DAIL-SQL高出15.5%。这些性能差异符合我们的预期，因为DAIL-SQL是专门为关系型数据库设计的，而空间数据库具有其独特的特性。SSpa方法及其变体（SSpa-SDBMS、SSpa-Geo和SSpa-Tips）通过将所使用的空间数据库管理、地理描述或模式提示的特定知识整合到模型的提示中，提高了性能，从而提升了其在空间数据库Text-to-SQL任务中的有效性。

接下来，我们分析 SSpa 和 DAIL-SQL 方法在数据集 1 和数据集 2 上的性能差异。如表 3 和表 4 所示，无论是使用 GPT-4 还是 DeepSeek，这两种方法（SSpa 和 DAIL-SQL）在所有场景（0-shot、1-shot、3-shot 和 5-shot）中，在数据集 1 上的表现始终优于数据集 2。该结果可归因于数据集 1 和数据集 2 之间 SQL 查询的差异。在数据集 1 中，由于存在派生列，问题可以通过通用 SQL 查询和空间 SQL 查询进行解决。然而，在数据集 2 中，由于移除了派生列，问题只能通过空间 SQL 查询来回答，而空间 SQL 查询通常比通用 SQL 查询更具挑战性。此外，与关系数据库相比，空间数据库提供的可用知识较少，这导致 GPT 模型在训练中利用的空间数据库知识减少，从而可能导致生成空间 SQL 查询的性能较差。

最后，我们评估了 SSpa 方法在各种场景下（0-shot、1-shot、3-shot 和 5-shot）的性能。如图 4 所示，对于 GPT-4 和 DeepSeek，SSpa 在非 0-shot 场景中均达到最佳性能，这表明在提示中加入相关示例可以提高 SQL 查询预测的准确性。然而，增加示例的数量并不总是能持续提高性能。例如，在 GPT-4 上，SSpa 在数据集 1 的 0-shot 中优于 1-shot，在数据集 2 的 3-shot 中优于 5-shot。前者可能是由于选择的单个示例不合适或不具代表性，而后者可能是 5-shot 场景中示例间存在干扰。相比之下，DeepSeek 并未出现这些异常，这可能是由于模型架构和训练的差异。不管怎样，GPT-4 和 DeepSeek 在非 0-shot 场景中都取得了最佳结果。

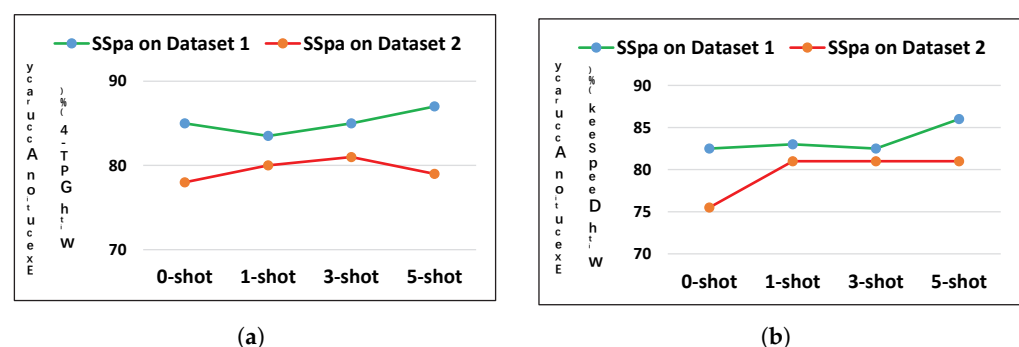


图 4. 一张折线图清晰地展示了 SSpa 在数据集 1 和数据集 2 上的 0-shot、1-shot、3-shot 和 5-shot 的表现。(a) SSpa 与 GPT-4 的表现。(b) SSpa 与 DeepSeek 的表现。

5.2. Effect of Each Component in the Prompt

As shown in Tables 3 and 4, our method, SSpa, outperforms SSpa-Tips, SSpa-SDBMS, and SSpa-Geo across all scenarios (0-shot, 1-shot, 3-shot, and 5-shot), whether using GPT-4 or DeepSeek. This indicates that removing any component from the prompt in our method negatively impacts its performance.

First, we investigate the role of the schema tips component in the prompts for the SSpa, SSpa-SDBMS, and SSpa-Geo methods. In our framework, if an exception occurs during the execution of a GPT-generated SQL query, we extract schema tips from the exception feedback. The extracted schema tips are then incorporated into the original prompt, and the augmented prompt is resubmitted to GPT for a new prediction. Hence, we analyze the number of initially failed queries that were corrected after the inclusion of schema tips. For example, a prompt for the question “What is the total area of the three northeastern provinces?” is sent to GPT, which generates the SQL query “Select Sum (Shape_Area) from cities where province in (‘辽宁省’, ‘吉林省’, ‘黑龙江省’)”. When executed, the SQL query produces an error “no such column: Shape_Area”. We extract the schema tip “The column ‘Shape_Area’ does not exist in the table ‘cities’ and should not be used.”, incorporate it into the original prompt, and resend it to GPT. After iterations, GPT produces the correct SQL query “Select Sum (Area (cities.Shape, 1)) from cities where province in (‘辽宁省’, ‘吉林省’, ‘黑龙江省’)”. Detailed data are presented in Tables 5 and 6.

We observe that, while SSpa-SDBMS with GPT-4 in the 1-shot scenario shows no improvement in correct SQL query predictions on Dataset 1, incorporating the schema tips component increases prediction accuracy in all other scenarios. Notably, with the schema tips component added to the prompt, our SSpa method accurately predicts up to eight SQL queries on both Dataset 1 and Dataset 2, representing 4% of the total 200 predictions. This indicates that in Text-to-SQL tasks for spatial databases, when SQL queries predicted by GPT encounter schema errors, such as missing tables or columns and ambiguous column names, extracting the schema tips and incorporating them into the prompt improves GPT’s SQL query prediction accuracy.

Table 5. Number of initially failed SQL query predictions corrected with the adding of the schema tips component by each method using GPT-4.

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
SSpa	1	2	2	8	1	3	7	8
SSpa-Tips	-	-	-	-	-	-	-	-
SSpa-SDBMS	2	0	3	3	1	3	4	4
SSpa-Geo	2	2	2	7	2	3	4	6

Table 6. Number of initially failed SQL query predictions corrected with the adding of the schema tips component by each method using DeepSeek.

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
SSpa	1	2	2	3	1	1	7	8
SSpa-Tips	-	-	-	-	-	-	-	-
SSpa-SDBMS	1	3	7	5	1	5	8	9
SSpa-Geo	2	4	3	5	1	2	7	12

Next, we examine the impact of the geographic descriptions component. The geographic descriptions component refers to the inclusion of geographic region descriptions for each table in the prompts of the SSpa method. As shown in Figure 3, the ‘airports’ table

5.2. Effect of Each Component in the Prompt

如表3和表4所示，我们的方法SSpa在所有场景（0-shot、1-shot、3-shot和5-shot）下，无论使用GPT-4还是DeepSeek，都优于SSpa-Tips、SSpa-SDBMS和SSpa-Geo。这表明在我们的方法中移除提示的任何组件都会对其性能产生负面影响。

首先，我们研究模式提示组件在 SSpa、SSpa-SDBMS 和 SSpa-Geo 方法提示中的作用。在我们的框架中，如果在执行 GPT 生成的 SQL 查询时发生异常，我们会从异常反馈中提取模式提示。提取的模式提示随后被纳入原始提示，并将增强后的提示重新提交给 GPT 以进行新的预测。因此，我们分析在包含模式提示后纠正的最初失败查询的数量。例如，对于问题“东北三省的总面积是多少？”的提示被发送给 GPT，GPT 生成的 SQL 查询为“Select Sum (Shape_Area) from cities where province in (‘辽宁省’ , ‘吉林省’ , ‘黑龙江省’)”。执行时，SQL 查询产生错误“no such column: Shape_Area”。我们提取模式提示“表 ‘cities’ 中不存在列 ‘Shape_Area’，不应使用”，将其纳入原始提示，并重新发送给 G

我们观察到，在1-shot场景下，使用GPT-4的SSpa-SDBMS在数据集1上的SQL查询正确预测没有提升，而在其他所有场景中，加入schema tips组件都提高了预测准确率。值得注意的是，当在提示中加入schema tips组件时，我们的SSpa方法在数据集1和数据集2上最多可以准确预测八条SQL查询，占总共200条预测的4%。这表明，在面向空间数据库的Text-to-SQL任务中，当GPT预测的SQL查询遇到模式错误（如缺少表或列、列名模糊）时，提取schema tips并将其融入提示中可以提高GPT的SQL查询预测准确率。

表5. 使用GPT-4时，通过每种方法添加模式提示组件纠正的最初失败SQL查询预测的数量。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
SSpa	1	2	2	8	1	3	7	8
SSpa-Tips	-	-	-	-	-	-	-	-
SSpa-SDBMS	2	0	3	3	1	3	4	4
SSpa-Geo	2	2	2	7	2	3	4	6

表 6. 使用 DeepSeek 时，每种方法通过添加模式提示组件纠正的最初失败的 SQL 查询预测数量。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
SSpa	1	2	2	3	1	1	7	8
SSpa-Tips	-	-	-	-	-	-	-	-
SSpa-SDBMS	1	3	7	5	1	5	8	9
SSpa-Geo	2	4	3	5	1	2	7	12

接下来，我们将研究地理描述组件的影响。地理描述组件是指在SSpa方法的提示中为每个表格包含地理区域描述。如图3所示，‘airports’表格

is described as “The ‘airports’ table records information about various airports in China.”. In contrast, the prompts for the SSpa-Geo and DAIL-SQL methods do not include such geographic descriptions. As shown in Table 2, 34 out of the 200 questions are related to geographic regions. Therefore, we evaluate the number of correct SQL query predictions for questions related to geographic regions across each method. The detailed results are presented in Tables 7 and 8.

We observe that SSpa consistently outperforms the DAIL-SQL method in the number of correct SQL query predictions across all scenarios. Except in the zero-shot scenario, where SSpa with GPT-4 slightly underperforms SSpa-Geo (SSpa accurately predicts 31 such questions compared to SSpa-Geo’s 32), SSpa surpasses SSpa-Geo in all other scenarios. This suggests that for region-specific questions, which frequently arise, incorporating geographic region descriptions into the prompt enhances the GPT model’s SQL query prediction accuracy.

Table 7. Number of correct SQL query predictions for region-related questions by each method using GPT-4.

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	9	8	10	11	6	7	11	11
SSpa	32	31	32	31	31	30	32	32
SSpa-Geo	27	16	13	13	32	17	12	11

Table 8. Number of correct SQL query predictions for region-related questions by each method using DeepSeek.

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	10	16	15	16	6	18	17	18
SSpa	31	33	33	33	31	34	33	33
SSpa-SDBMS	13	21	23	24	10	23	31	31
SSpa-Geo	29	23	19	19	28	22	20	18
SSpa-Tips	30	31	31	31	27	32	31	30

Finally, we analyze the role of the SDBMS knowledge component. Compared to the SSpa-SDBMS method, the SSpa method incorporates SDBMS knowledge, specifically descriptions of certain functions, into its prompt. Consequently, we evaluate the number of initially failed SQL query predictions by SSpa-SDBMS that were successfully corrected by incorporating the SDBMS knowledge component (i.e., the SSpa method), with detailed data presented in Table 9.

We observe that across all scenarios (0-shot, 1-shot, 3-shot, and 5-shot), a significant number of initially failed SQL query predictions are successfully corrected after incorporating the SDBMS knowledge component into the prompt. The number of such successful corrections is at least 19 (9.5% of the total 200 queries) for GPT-4 and 14 (7%) for DeepSeek in the 5-shot scenario. This can be attributed to the lack of knowledge about the spatial database used in SSpa-SDBMS. Since spatial database management systems vary in type, each with different functions, the SQL queries generated by SSpa-SDBMS are more likely to invoke functions that are incompatible with the used spatial database management system. For example, we look into the prompts generated by the SSpa-SDBMS and SSpa methods with GPT-4 in the 5-shot scenario for the question “What is the length of the Yangtze River within Hubei Province?”. Neither prompt includes examples of the function GLength (c Curve, 1). The SSpa-SDBMS’ prompt lacks the SDBMS knowledge, whereas the SSpa’ prompt incorpo-

被描述为“‘airports’表记录了中国各个机场的信息”。相比之下，SSpa-Geo 和 DAIL-SQL 方法的提示中并不包含这样的地理描述。如表 2 所示，200 个问题中有 34 个与地理区域相关。因此，我们评估每种方法在与地理区域相关的问题上正确 SQL 查询预测的数量。详细结果见表 7 和表 8。

我们观察到，在所有场景中，SSpa 在正确 SQL 查询预测数量上始终优于 DAIL-SQL 方法。唯一的例外是零样本场景，在该场景下，使用 GPT-4 的 SSpa 略逊于 SSpa-Geo（SSpa 正确预测了 31 个此类问题，而 SSpa-Geo 为 32 个），在其他所有场景中，SSpa 都超过了 SSpa-Geo。这表明，对于经常出现的特定区域问题，在提示中加入地理区域描述可以提高 GPT 模型的 SQL 查询预测准确性。

表7. 使用 GPT-4 时，各方法针对区域相关问题的正确 SQL 查询预测数量。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	9	8	10	11	6	7	11	11
SSpa	32	31	32	31	31	30	32	32
SSpa-Geo	27	16	13	13	32	17	12	11

表 8. 使用 DeepSeek 时各方法对区域相关问题的正确 SQL 查询预测数量。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	10	16	15	16	6	18	17	18
SSpa	31	33	33	33	31	34	33	33
SSpa-SDBMS	13	21	23	24	10	23	31	31
SSpa-Geo	29	23	19	19	28	22	20	18
SSpa-Tips	30	31	31	31	27	32	31	30

最后，我们分析了SDBMS知识组件的作用。与SSpa-SDBMS方法相比，SSpa方法在其提示中加入了SDBMS知识，具体包括对某些函数的描述。因此，我们评估了通过引入SDBMS知识组件（即SSpa方法）成功纠正的、SSpa-SDBMS最初预测失败的SQL查询的数量，详细数据见表9。

我们观察到，在所有场景（零示例、单示例、三示例和五示例）中，大量最初失败的 SQL 查询预测在将 SDBMS 知识组件纳入提示后得到了成功纠正。在五示例场景中，这类成功纠正的数量至少为 19（占总共 200 个查询的 9.5%）对于 GPT-4，而 DeepSeek 为 14（7%）。这可以归因于对 SSpa-SDBMS 所使用的空间数据库缺乏了解。由于空间数据库管理系统的类型各异，每种类型具有不同的功能，SSpa-SDBMS 生成的 SQL 查询更可能调用与所使用的空间数据库管理系统不兼容的函数。例如，我们查看了在五示例场景中，针对问题“长江在湖北省的长度是多少？”由 SSpa-SDBMS 和 SSpa 方法使用 GPT-4 生成的提示。两个提示都未包含函数 GLength(c Curve, 1) 的示例。SSpa-SDBMS 的

rats the SDBMS knowledge, including an explanation of the GLength (c Curve, 1) function. For this question, the SSpa method produces a correct SQL query “Select Sum (GLength (Intersection (rivers.shape, provinces.shape), 1)) AS LengthInHubei from rivers inner join provinces on Intersects (rivers.shape, provinces.shape) = 1 where rivers.name = ‘长江’ and provinces.name = ‘湖北省’”. In contrast, the SSpa-SDBMS method generates an incorrect SQL query “Select Sum (GLength (Intersection (rivers.shape, provinces.shape))) AS Length from rivers join provinces on Intersects (rivers.shape, provinces.shape) where rivers.name = ‘长江’ and provinces.name = ‘湖北省’”. This query incorrectly uses the GLength (c Curve) function without the required parameter 1. Notably, despite the absence of examples for the GLength (c Curve) or GLength (c Curve, 1) functions in the SSpa-SDBMS prompt, the method still generates a query using GLength (c Curve), albeit incorrectly. This suggests that the inclusion of the GLength (c Curve, 1) explanation in the SSpa prompt effectively can guide GPT to use the correct function format. However, incorporating the SDBMS knowledge into the prompt can occasionally lead to errors in previously correct predictions. Overall, adding the SDBMS knowledge to the prompt significantly enhances the accuracy of GPT’s SQL query predictions.

Table 9. Number of initially failed SQL query predictions by SSpa-SDBMS corrected with the SDBMS knowledge component.

Model	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
GPT-4	73	57	49	36	86	66	35	19
DeepSeek	66	38	32	21	78	35	17	14

6. Limitations

There are also several limitations in our work.

First, in the SDBMS knowledge component of the prompt, we manually specify only a few commonly used functions. The accuracy of the generated SQL queries could likely be improved if more relevant functions, tailored to each specific question, were provided. Therefore, in future work, we will automatically select and include appropriate functions in the prompt based on the target question.

Second, in the schema tips component of the prompt, we address only three scenarios: non-existent tables or columns and column name ambiguities. By extracting a broader range of schema tips from the exception feedback and incorporating them into the prompt, we believe the accuracy of the generated SQL queries could be further enhanced.

Finally, due to the inherent difficulty in constructing spatial questions and their corresponding SQL queries, our dataset remains limited. As a result, we have identified only a few patterns, such as adding geographic region descriptions for tables. With a larger dataset and a more diverse set of question types, we believe additional patterns more suitable for constructing prompts for spatial databases could be discovered. Of course, all three of these limitations represent areas for future work and can be gradually addressed over time.

7. Conclusions

Text-to-SQL for spatial databases enables the translation of natural language questions into corresponding SQL queries based on a given database schema, making spatial data more accessible to non-experts. This capability has garnered increasing attention from researchers. In our study, we propose a GPT-based method for constructing prompts to solve the Text-to-SQL task for spatial databases. Our method introduces three key innovations: the integration of SDBMS knowledge, geographic descriptions, and schema

涉及SDBMS知识，包括对GLength(c Curve, 1)函数的解释。对于这个问题，SSpa方法生成了正确的SQL查询“Select Sum(GLength(Intersection(rivers.shape, provinces.shape), 1)) AS LengthInHubei from rivers inner join provinces on Intersects(rivers.shape, provinces.shape) = 1 where rivers.name = ‘长江’ and provinces.name = ‘湖北省’”。相比之下，SSpa-SDBMS方法生成了一个错误的SQL查询“Select Sum(GLength(Intersection(rivers.shape, provinces.shape))) AS Length from rivers join provinces on Intersects(rivers.shape, provinces.shape) where rivers.name = ‘长江’ and provinces.name = ‘湖北省’”。该查询错误地使用了GLength(c Curve)函数而没有提供所需的参数1。值得注意的是，尽管在SSpa-SDBMS提示中没有关于GLength(c Curve)或GLength(c Curve, 1)函数的示例，该方法仍然生成了使用GLength(c Curve)的查询，尽管是错误的。这表明包括

表9. 通过SDBMS知识组件纠正的SSpa-SDBMS最初失败的SQL查询预测数量。

Model	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
GPT-4	73	57	49	36	86	66	35	19
DeepSeek	66	38	32	21	78	35	17	14

6. 限制

我们的工作也存在一些局限性。

首先，在提示的SDBMS知识组件中，我们仅手动指定了一些常用函数。如果提供更多与每个具体问题相关的函数，生成的SQL查询的准确性可能会得到提高。因此，在未来的工作中，我们将根据目标问题自动选择并在提示中包含适当的函数。

其次，在提示的模式提示组件中，我们只处理三种情况：不存在的表或列以及列名歧义。通过从异常反馈中提取更广泛的模式提示并将其纳入提示中，我们相信生成的 SQL 查询的准确性可以进一步提高。

最后，由于构建空间问题及其相应 SQL 查询的固有困难，我们的数据集仍然有限。因此，我们只识别出少数模式，例如为表添加地理区域描述。我们相信，随着数据集的增大和问题类型的更加多样化，可以发现更多更适合构建空间数据库提示的模式。当然，这三种局限性都代表了未来的研究方向，并且可以随着时间的推移逐步解决。

7. 结论

面向空间数据库的文本到 SQL 转换可以将自然语言问题翻译为基于给定数据库模式的相应 SQL 查询，从而使非专家更容易访问空间数据。这一能力已经引起了研究人员越来越多的关注。在我们的研究中，我们提出了一种基于 GPT 的方法，用于构建提示以解决空间数据库的文本到 SQL 任务。我们的方法引入了三项关键创新：空间数据库管理系统（SDBMS）知识的整合、地理描述以及数据库模式（schema）。

tips within the prompt. We detail the methodology and present systematic experiments. The results demonstrate that all three components, including SDBMS knowledge, geographic descriptions, and schema tips, are essential for optimal performance. Omitting any of these components adversely affects the accuracy of the method. Additionally, to further advance Text-to-SQL for spatial databases, we have created a bilingual dataset that supports both Chinese and English. Our work has the potential to greatly enhance the accessibility of spatial databases for non-experts, facilitating their use and application.

Author Contributions: Conceptualization, Hui Wang and Li Guo; methodology, Hui Wang and Li Guo; software, Le Liu and Hui Wang; data curation, Le Liu; writing—original draft preparation, Hui Wang and Li Guo; writing—review and editing, Hui Wang, Li Guo, Yubin Liang, and Jiajin Huang; supervision, Hui Wang and Li Guo; funding acquisition, Hui Wang. All authors have read and agreed to the published version of the manuscript.

Funding: This work is supported by the Doctoral Fund of Tianjin Normal University (Grant No. 52XB1619) and the Open Fund of Tianjin Geospatial Information Technology Engineering Center.

Data Availability Statement: The data that support the findings of this study are openly available in GitHub at <https://github.com/beta512/SpatialSQL> (accessed on 14 July 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Leszczynski, A.; Crampton, J. Introduction: Spatial Big Data and Everyday Life. *Big Data Soc.* **2016**, *3*, 1–6. [\[CrossRef\]](#)
2. Liu, M.; Wang, X.; Xu, J. NALSD: A Natural Language Interface for Spatial Databases. In Proceedings of the 18th International Symposium on Spatial and Temporal Data, Calgary, AB, Canada, 23–25 August 2023; pp. 175–179.
3. Liu, M.; Wang, X.; Xu, J.; Lu, H. NALSpatial: An Effective Natural Language Transformation Framework for Queries over Spatial Data. In Proceedings of the 31st ACM International Conference on Advances in Geographic Information Systems, Hamburg, Germany, 13–16 November 2023; p. 4.
4. Wang, X.; Liu, M.; Xu, J.; Lu, H. NALMO: Transforming Queries in Natural Language for Moving Objects Databases. *GeoInformatica* **2023**, *27*, 427–460. [\[CrossRef\]](#)
5. Liu, J.; Shen, D.; Zhang, Y.; Dolan, B.; Carin, L.; Chen, W. What Makes Good In-Context Examples for GPT-3? In Proceedings of the 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures, Dublin, Ireland, 26–27 May 2022; pp. 100–114.
6. Guo, C.; Tian, Z.; Tang, J.; Wang, P.; Wen, Z.; Yang, K.; Wang, T. Prompting GPT-3.5 for Text-to-SQL with De-semanticization and Skeleton Retrieval. In Proceedings of the Trends in Artificial Intelligence—20th Pacific Rim International Conference on Artificial Intelligence, Jakarta, Indonesia, 15–19 November 2023; pp. 262–274.
7. Pourreza, M.; Rafiei, D. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In Proceedings of the 37th International Conference on Neural Information Processing Systems, New Orleans, LA, USA, 10–16 December 2024; pp. 36339–36348.
8. Zelle, J.M.; Mooney, R.J. Learning to Parse Database Queries using Inductive Logic Programming. In Proceedings of the Thirteenth National Conference on Artificial Intelligence, Portland, OR, USA, 4–8 August 1996; pp. 1050–1055.
9. Tang, L.R.; Mooney, R.J. Automated Construction of Database Interfaces: Integrating Statistical and Relational Learning for Semantic Parsing. In Proceedings of the 2000 Joint SIGDAT Conference on Empirical Methods in Natural Language Processing and Very Large Corpora, Hong Kong, China, 7–8 October 2000; pp. 133–141.
10. Iyer, S.; Konstas, I.; Cheung, A.; Krishnamurthy, J.; Zettlemoyer, L. Learning a Neural Semantic Parser from User Feedback. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, Vancouver, BC, Canada, 30 July–4 August 2017; pp. 963–973.
11. Zhong, V.; Xiong, C.; Socher, R. Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning. *arXiv* **2017**, arXiv:1709.00103. [\[CrossRef\]](#)
12. Yu, T.; Zhang, R.; Yang, K.; Yasunaga, M.; Wang, D.; Li, Z.; Ma, J.; Li, I.; Yao, Q.; Roman, S.; et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 31 October–4 November 2018; pp. 3911–3921.

提示中的技巧。我们详细说明了方法论并呈现了系统实验。结果表明，包括SDBMS知识、地理描述和模式提示在内的所有三个组成部分，对于达到最佳性能都是必不可少的。省略任何一个组成部分都会对方法的准确性产生负面影响。此外，为进一步推进面向空间数据库的Text-to-SQL研究，我们创建了一个支持中英文的双语数据集。我们的工作有可能显著提高非专业人士使用空间数据库的便利性，促进其使用和应用。

作者贡献：概念构思，王辉和郭力；方法学，王辉和郭力；软件，刘乐和王辉；数据整理，刘乐；原稿撰写，王辉和郭力；审阅与编辑，王辉、郭力、梁宇斌和黄佳进；监督，王辉和郭力；资金获取，王辉。所有作者已阅读并同意已发表版本的手稿。

资金：本研究得到天津师范大学博士基金（项目编号52XB1619）及天津地理空间信息技术工程中心开放基金的资助。

数据可用性声明：支持本研究结果的数据可公开获取于 GitHub，链接为 <https://github.com/beta512/SpatialSQL>（访问日期：2025年7月14日）。

利益冲突：作者声明没有利益冲突

成立

参考文献

1. Leszczynski, A.; Crampton, J. 引言：空间大数据与日常生活。 *Big Data Soc.* 2016, 3, 1–6. [CrossRef]
2. Liu, M.; Wang, X.; Xu, J. NALSD：用于空间数据库的自然语言接口。发表于第18届国际空间与时间数据研讨会论文集，加拿大卡尔加里，2023年8月23–25日；第175–179页。
3. Liu, M.; Wang, X.; Xu, J.; Lu, H. NALSpatial：用于空间数据查询的有效自然语言转换框架。发表于第31届ACM地理信息系统国际会议论文集，德国汉堡，2023年11月13–16日；第4页。
4. Wang, X.; Liu, M.; Xu, J.; Lu, H. NALMO：针对移动对象数据库的自然语言查询转换。 *GeoInformatica* 2023, 27, 427–460. [CrossRef]
5. Liu, J.; Shen, D.; Zhang, Y.; Dolan, B.; Carin, L.; Chen, W. 什么样的示例适合作为GPT-3的上下文？发表于第3届深度学习架构知识抽取与整合研讨会论文集，都柏林，

13. Min, Q.; Shi, Y.; Zhang, Y. A Pilot Study for Chinese SQL Semantic Parsing. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, Hong Kong, China, 3–7 November 2019; pp. 3652–3658.
14. Wang, L.; Zhang, A.; Wu, K.; Sun, K.; Li, Z.; Wu, H.; Zhang, M.; Wang, H. DuSQL: A Large-Scale and Pragmatic Chinese Text-to-SQL Dataset. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, Online, 16–20 November 2020; pp. 6923–6935.
15. Gan, Y.; Chen, X.; Huang, Q.; Purver, M.; Woodward, J.R.; Xie, J.; Huang, P. Towards Robustness of Text-to-SQL Models against Synonym Substitution. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, Online, 1–6 August 2021; pp. 2505–2515.
16. Gan, Y.; Chen, X.; Purver, M. Exploring Underexplored Limitations of Cross-Domain Text-to-SQL Generalization. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Punta Cana, Dominican Republic, 7–11 November 2021; pp. 8926–8931.
17. Shaw, P.; Chang, M.-W.; Pasupat, P.; Toutanova, K. Compositional Generalization and Natural Language Variation: Can a Semantic Parsing Approach Handle Both? In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, Online, 1–6 August 2021; pp. 922–938.
18. Li, J.; Hui, B.; Qu, G.; Yang, J.; Li, B.; Li, B.; Wang, B.; Qin, B.; Geng, R.; Huo, N.; et al. Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs. In Proceedings of the 37th International Conference on Neural Information Processing Systems, New Orleans, LA, USA, 10–16 December 2023; pp. 42330–42357.
19. Sutskever, I.; Vinyals, O.; Le, Q.V. Sequence to Sequence Learning with Neural Networks. In Proceedings of the 27th International Conference on Neural Information Processing Systems, Montreal, QC, Canada, 8–13 December 2014; pp. 3104–3112.
20. Guo, J.; Zhan, Z.; Gao, Y.; Xiao, Y.; Lou, J.-G.; Liu, T.; Zhang, D. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019; pp. 4524–4535.
21. Choi, D.; Shin, M.C.; Kim, E.; Shin, D.R. RYANSQL: Recursively Applying Sketch-Based Slot Fillings for Complex Text-to-SQL in Cross-Domain Databases. *Comput. Linguist.* **2021**, *47*, 309–332. [\[CrossRef\]](#)
22. Hwang, W.; Yim, J.; Park, S.; Seo, M. A Comprehensive Exploration on WikiSQL with Table-Aware Word Contextualization. In Proceedings of the 33rd International Conference on Neural Information Processing Systems, Vancouver, BC, Canada, 8–14 December 2019.
23. Yin, P.; Neubig, G.; Yih, W.-t.; Riedel, S. TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 8413–8426.
24. Herzig, J.; Nowak, P.K.; Müller, T.; Piccinno, F.; Eisenschlos, J. TAPAS: Weakly Supervised Table Parsing via Pre-Training. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 4320–4333.
25. Yu, T.; Wu, C.-S.; Lin, X.V.; Wang, B.; Tan, Y.C.; Yang, X.; Radev, D.R.; Socher, R.; Xiong, C. Grappa: Grammar-Augmented Pre-Training for Table Semantic Parsing. In Proceedings of the 9th International Conference on Learning Representations, Online, 3–7 May 2021.
26. Li, J.; Wang, W.; Ku, W.-S.; Tian, Y.; Wang, H. SpatialNLI: A spatial Domain Natural Language Interface to Databases Using Spatial Comprehension. In Proceedings of the 27th ACM International Conference on Advances in Geographic Information Systems, Chicago, IL, USA, 5–8 November 2019; p. 10.
27. Wang, W.; Li, J.; Ku, W.-S.; Wang, H. Multilingual Spatial Domain Natural Language Interface to Databases. *GeoInformatica* **2024**, *28*, 29–52. [\[CrossRef\]](#)
28. Hong, Z.; Yuan, Z.; Zhang, Q.; Chen, H.; Dong, J.; Huang, F.; Huang, X. Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL. *arXiv* **2024**, arXiv:2406.08426.
29. Liu, X.; Shen, S.; Li, B.; Ma, P.; Jiang, R.; Zhang, Y.; Fan, J.; Li, G.; Tang, N.; Luo, Y. A Survey of NL2SQL with Large Language Models: Where Are We, and Where Are We Going? *arXiv* **2024**, arXiv:2408.05109. [\[CrossRef\]](#)
30. Shi, L.; Tang, Z.; Zhang, N.; Zhang, X.; Yang, Z. A Survey on Employing Large Language Models for Text-to-SQL Tasks. *arXiv* **2024**, arXiv:2407.15186. [\[CrossRef\]](#)
31. Dong, X.; Zhang, C.; Ge, Y.; Mao, Y.; Gao, Y.; Chen, L.; Lin, J.; Lou, D. C3: Zero-Shot Text-to-SQL with ChatGPT. *arXiv* **2023**, arXiv:2307.07306.
32. Zhang, H.; Cao, R.; Chen, L.; Xu, H.; Yu, K. ACT-SQL: In-Context Learning for Text-to-SQL with Automatically-Generated Chain-of-Thought. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 6–10 December 2023; pp. 3501–3532.
33. Gao, D.; Wang, H.; Li, Y.; Sun, X.; Qian, Y.; Ding, B.; Zhou, J. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. Vldb Endow.* **2024**, *17*, 1132–1145. [\[CrossRef\]](#)

13. Min, Q.; Shi, Y.; Zhang, Y. 一项针对中文 SQL 语义解析的初步研究。载于《2019 年自然语言处理实证方法会议暨第 9 届国际联合自然语言处理会议论文集》，香港，中国，2019 年 11 月 3–7 日；页 3652–3658。
14. Wang, L.; Zhang, A.; Wu, K.; Sun, K.; Li, Z.; Wu, H.; Zhang, M.; Wang, H. DuSQL: 大规模且实用的中文文本到 SQL 数据集。载于《2020 年自然语言处理实证方法会议论文集》，线上，2020 年 11 月 16–20 日；页 6923–6935。
15. Gan, Y.; Chen, X.; Huang, Q.; Purver, M.; Woodward, J.R.; Xie, J.; Huang, P. 提升文本到 SQL 模型对同义词替换的鲁棒性。载于《第 59 届计算语言学协会年会暨第 11 届国际联合自然语言处理会议论文集》，线上，2021 年 8 月 1–6 日；页 2505–2515。
16. Gan, Y.; Chen, X.; Purver, M. 探索尚未充分研究的局限性

34. Wu, H.; Shen, Z.; Hou, S.; Liang, J.; Jiao, H.; Qing, Y.; Zhang, X.; Li, X.; Gui, Z.; Guan, X.; et al. AutoGEEval: A Multimodal and Automated Evaluation Framework for Geospatial Code Generation on GEE with Large Language Models. *ISPRS Int. J.-Geo-Inf.* **2025**, *14*, 256. [[CrossRef](#)]
35. Hou, S.; Jiao, H.; Shen, Z.; Liang, J.; Zhao, A.; Zhang, X.; Wang, J.; Wu, H. Chain-of-Programming (CoP): Empowering Large Language Models for Geospatial Code Generation Task. *Int. J. Digit. Earth* **2025**, *18*, 2509812. [[CrossRef](#)]
36. Hou, S.; Liang, J.; Zhao, A.; Wu, H. GEE-OPs: An Operator Knowledge Base for Geospatial Code Generation on the Google Earth Engine Platform Powered by Large Language Models. *Geo-Spat. Inf. Sci.* **2025**, 1–22. [[CrossRef](#)]
37. Hou, S.; Shen, Z.; Zhao, A.; Liang, J.; Gui, Z.; Guan, X.; Li, R.; Wu, H. GeoCode-GPT: A Large Language Model for Geospatial Code Generation. *Int. J. Appl. Earth Obs. Geoinf.* **2025**, *138*, 104456. [[CrossRef](#)]
38. Hou, S.; Zhao, A.; Liang, J.; Shen, Z.; Wu, H. Geo-FuB: A Method for Constructing An Operator-Function Knowledge Base for Geospatial Code Generation with Large Language Models. *Knowl.-Based Syst.* **2025**, *319*, 113624. [[CrossRef](#)]
39. Jiang, Y.; Yang, C. Is ChatGPT a Good Geospatial Data Analyst? Exploring the Integration of Natural Language into Structured Query Language within a Spatial Database. *ISPRS Int. J.-Geo-Inf.* **2024**, *13*, 26. [[CrossRef](#)]
40. Nan, L.; Zhao, Y.; Zou, W.; Ri, N.; Tae, J.; Zhang, E.; Cohan, A.; Radev, D. Enhancing Text-to-SQL Capabilities of Large Language Models: A Study on Prompt Design Strategies. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 6–10 December 2023; pp. 14935–14956.
41. Chang, S.; Fosler-Lussier, E. How to Prompt LLMs for Text-to-SQL: A Study in Zero-Shot, Single-Domain, and Cross-Domain Settings. *arXiv* **2023**, arXiv:2305.11853.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

34. 吴华; 沈志; 侯顺; 梁杰; 焦浩; 庆阳; 张翔; 李昕; 桂志; 关祥; 等。AutoGEEval: 一个用于GEE上地理空间代码生成的多模态自动化评估框架, 结合大型语言模型。 *ISPRS Int. J.-Geo-Inf.* 2025, 14, 256. [CrossRef]35. 侯顺; 焦浩; 沈志; 梁杰; 赵安; 张翔; 王健; 吴华。Chain-of-Programming (CoP): 赋能大型语言模型进行地理空间代码生成任务。 *Int. J. Digit. Earth* 2025, 18, 2509812. [CrossRef]36. 侯顺; 梁杰; 赵安; 吴华。GEE-OPs: 一个基于操作符的知识库, 用于在谷歌地球引擎平台上进行地理空间代码生成, 依托大型语言模型。 *Geo-Spat. Inf. Sci.* 2025, 1–22. [CrossRef]37. 侯顺; 沈志; 赵安; 梁杰; 桂志; 关祥; 李锐; 吴华。GeoCode-GPT: 一个用于地理空间代码生成的大型语言模型。 *Int. J. Appl. Earth Obs. Geoinf.* 2025, 138, 104456. [CrossRef]38. 侯顺; 赵安; 梁杰; 沈志; 吴华。Geo-FuB: 一种构建地理操作符-函数知识库的方法

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.